

Orchestra Mode – Master Engineering Specification V1.0

1. Introduction & Design Intent

Orchestra Mode is a harmonically-aware spatial bloom engine for the Big Pi / OpenInfinity reverb platform. It is inspired by the Cloudburst Ensemble-mode concept, where harmonic voices emerge organically from within the reverb tail rather than existing as a shimmer feedback loop or external synth layer. The design objective is to make the reverb space itself behave like a dynamically generated ensemble. Harmonic material must feel embedded within the spatial field, respond to performance dynamics, and remain musically stable under both clean and high-gain input conditions.

2. Signal Architecture

Signal Flow: Input → Conditioning → PreDelay → Early Diffusion → Orchestra Injection → Late Tank/FDN → Tone/Damping → Mix → Output
Critical Requirement: Injection MUST occur after Early Diffusion and before the Late Tank matrix to ensure harmonic smearing occurs naturally within the spatial domain.

3. Harmonic Analysis Engine

- FFT Size: 2048 preferred (1024 fallback) at 48kHz.
- Hop Size: 512 samples.
- Convert FFT magnitude spectrum into 12-bin chroma representation.
- Chroma smoothing window: 100–200 ms.
- Root selection via dominant chroma bin with hysteresis (≥ 3 dB dominance).
- Major/minor inference via third interval strength comparison.
- Confidence gating to prevent unstable detection under noise.

4. Voice Engine Design

- Voice Count: 4–8 bandlimited oscillators (default 6).
- Intervals: Root (0), Fifth (+7), Octave (+12), Third (+3/+4 gated), Low Octave (-12), Detuned duplicate.
- Independent LFO drift (± 1 –4 cents).
- Pan distribution for stereo width.

- Attack: 150–600 ms (scaled by Size).
- Release: 1–6 s (scaled by Decay).
- Low-pass filtering: 800–6000 Hz (Tone mapped).

5. Envelope & Bloom System

Envelope Follower:

Attack: 10 ms

Release: 250 ms

Bloom Integrator:

$\text{releaseCoef} = \exp(-1 / (\text{releaseTime} * \text{sampleRate}))$

$\text{bloom}[n] = \max(\text{env}[n], \text{bloom}[n-1] * \text{releaseCoef})$

Voice Gain:

$\text{voiceGain} = \text{OrchestraAmount} * \text{smoothstep}(\text{thresholdLow}, \text{thresholdHigh}, \text{bloom})$

6. Injection Path & Protection

$\text{orchestraInjection} = \text{OrchestraEngine.process}(\text{input})$

Soft Clip Protection:

$y = \tanh(x * 1.2)$

$\text{excite} = \text{early} + \text{softLimitedInjection}$

$\text{tankOut} = \text{Tank.process}(\text{excite})$

7. Parameter Mapping

- Mix: Dry/Wet balance.
- Decay: Controls RT60 and voice release scaling.
- PreDelay: Standard pre-delay.
- Size: Tank size + bloom attack inertia.
- ModDepth: Detune spread + stereo width.
- ModRate: Ensemble drift rate.
- Tone: Reverb damping + voice LPF cutoff.
- Orchestra Amount: Harmonic injection intensity.

8. Code Implementation Strategy

Modular Implementation:

Files:

- OrchestraEngine.h / OrchestraEngine.cpp

Optional:

- OrchestraHarmony module
- OrchestraVoices module

Integration inside ReverbProcessor:

```
early = EarlyDiffusion.process(input)
injection = orchestra.process(input, params)
excite = early + injection
tankOut = Tank.process(excite)
```

Requirements:

- OrchestraAmount = 0 must null to base reverb output.
- No per-sample dynamic memory allocation.
- Reset buffers on mode or sample-rate change.
- Full parameter smoothing across all mapped controls.

9. CPU & Performance Targets

- Target Platform: Raspberry Pi 5 at 48kHz.
- Block-based FFT processing.
- Maximum 6–8 active voices.
- No dynamic allocations inside audio callback.
- Injection must not destabilize tank feedback structure.

10. Validation & Testing

- Null test with OrchestraAmount = 0.
- Major/minor stability across chord transitions.
- Noise resilience under high gain.
- Overload protection at max wet + injection.
- Fast passage harmonic smoothing validation.